

Three Adjacencies: Why CKS Is Not RAG, Not Parametric Memory, and Not External Structured Memory

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 24 April 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the boundaries between CKS and the three adjacent design objects with which it is most often conflated, so that downstream work can adopt, compose, or argue against the pattern without collapsing it into one of its neighbors.

Abstract

The Coordination Knowledge Substrate (CKS) design pattern is most often conflated with three adjacent design objects: retrieval-augmented generation (RAG), parametric (in-weight) memory, and external structured memory in the style of Knowledge Objects (KO) and OIDA. The source paper addresses each adjacency in compact disambiguations (§1.2, §2.3) and develops the load-bearing differentiations at §3.1, §5.2, §6.1, §6.2, and §8.2. This note assembles those boundaries into a single operational reference. For each adjacent object, the note states what the object is, the architectural commitment that distinguishes CKS from it, the most common conflation pattern, and a one-sentence test for distinguishing CKS from the adjacent object in any specific instance. A closing section explains why naming the boundaries matters: hybrid systems in which CKS is composed with one or more of the adjacent objects are coherent only when the layers are named explicitly, and the boundaries this note formalizes are what makes that explicit naming possible.

1. Why this note exists

The CKS source paper names five disambiguations in §1.2 — CKS is not RAG, not parametric memory, not a multi-agent protocol, not a governance platform, not a complete system architecture — and elaborates them in §2.3. Three of these adjacencies cluster around a single underlying axis: the architectural location of memory or knowledge state relative to the LLM. RAG locates content in a retrieval index the LLM reads at inference time. Parametric memory locates content in the model weights themselves. External structured memory locates content in a typed external store the LLM reads as a substrate client. CKS locates coordination knowledge, governance semantics, and conflict-handling logic in a persistent, human-governed substrate outside the LLM — where the

substrate is a coordination artifact humans hold authority over rather than a retrieval target, a parameter set, or a substrate client's data store.

Throughout this note, “outside the model” carries the meaning the source paper assigns it at §1.2: the architectural location of coordination, governance, and decision state in a persistent, human-governed substrate, rather than the narrower parametric-vs-non-parametric distinction standard in retrieval-augmented generation and external-memory work. This wider sense is what makes CKS distinct from each of the three adjacencies that take “outside the model” in the narrower sense.

A fourth memory pattern — per-session in-context memory, where the working state of a session lives in the LLM's context window — is the failure mode that external structured memory is designed to address. The source paper uses Knowledge Objects' “context rot” vocabulary (§6.2) to name the failure: capacity overflow, compaction loss, and goal drift under repeated compression passes. In-context memory is not a separate top-level adjacency in this note because it is not a candidate substitute for what CKS provides; it is what every external-memory pattern, CKS included, is designed not to be.

2. CKS is not RAG

What RAG is. Retrieval-augmented generation is an architectural pattern in which an LLM, at inference time, queries an external index over a corpus of source documents (or chunks thereof) and uses retrieved content as additional context for generation. The index is typically built by embedding documents into a vector space and retrieving by similarity, though graph and taxonomy variants (GraphRAG, CogGRAG, TaSR-RAG) place structured indices in front of retrieval. The primary design goal is retrieval quality — getting the right source content into the LLM's context window — and the design object is a corpus-plus-index pair. (Vector databases are infrastructure that RAG runs on; they are not themselves RAG. The RAG pattern is the architectural use of an index over a corpus to ground generation, regardless of whether the index lives in a vector store, a graph store, or a flat file.)

The CKS-distinguishing commitment. In CKS, the substrate is a coordination artifact humans read from, write into, and hold authority over (§§1.2, 2.3, 3.1). Its primary function is to encode decisions, rationale, authority, and preserved contradictions, not to retrieve source material. It is not an index over a corpus, and it does not exist primarily to be queried at LLM inference time. The substrate carries content humans authored or governed, not content extracted from prior text and recovered by similarity match. The design object is the substrate itself, not a corpus-plus-index pair.

The conflation pattern. The most common reading collapses CKS into “RAG with governance metadata” — a retrieval index over a corpus, with policy, access control, and provenance attached to retrieved units. This reading mistakes governance attached to retrieval for governance located in a coordination artifact. RAG-with-governance improves what the LLM reads (§1.1); it does not turn what was decided, by whom, and why into an addressable artifact the next participant can inspect and extend (§1.1). The misreading therefore loses the entire coordination-and-decision-layer move (§2.1) that distinguishes CKS from asset-layer work in data governance and enterprise knowledge bases.

One-sentence test. If the architecture’s primary function is to retrieve source material into LLM context at inference time, it is RAG; if the primary function is to carry decisions, rationale, and preserved contradictions as addressable content humans hold authority over, it is CKS.

3. CKS is not parametric memory

What parametric memory is. Parametric memory — also called in-weight memory — is the memory that lives in a model’s weights and is modified through training: full pretraining, continual pretraining, fine-tuning (full or LoRA), or knowledge editing techniques such as ROME, MEMIT, and MEND. The unit of storage is a parameter; the unit of update is a gradient step; the cost of expansion bends superlinearly at small edit counts. The source paper at §6.2 surveys mechanistic evidence for the bend: sequential fine-tuning severely disrupts 15–23% of attention heads in lower layers, ROME begins catastrophic forgetting between 100 and 1,000 edits depending on settings, and MEND degrades toward zero F1 before 100. The design object is the model’s weights themselves.

The CKS-distinguishing commitment. In CKS, the substrate is external to the LLM and deterministic, and its state changes through human-governed writes rather than through training (§2.3). Substrate content is added by human authorship, by LLM drafting under human authority, or by automated cells operating under human-authored orchestration rules — never by gradient updates to the model. The cost model is database-like: storage grows linearly with content (§6.1), inheriting the empirical cost curve from external-substrate work most directly demonstrated by Knowledge Objects (§6.2). The substrate is not the model; the model does not change as the substrate grows.

The conflation pattern. The most common reading takes “memory” in CKS to mean the same kind of thing as “memory” in continual learning or knowledge editing — knowledge stored somewhere in the system that the model uses to answer. This reading mistakes the location of storage. Parametric memory lives inside the model and is recovered through forward passes over weight-encoded content; CKS lives outside the model and is recovered through reads against an addressable substrate. The cost-model consequence of the misreading is severe: parametric expansion is superlinear and frontier-bending (§6.2), while CKS expansion is linear-in-additions (§6.1). Treating CKS as a parametric pattern imports cost-curve assumptions the architecture does not have.

One-sentence test. If expanding the system’s knowledge requires modifying model weights through any training procedure, the memory is parametric; if expansion happens through writes to an addressable external store under human authority, it is not.

4. CKS is not external structured memory (Knowledge Objects, OIDA, and similar)

What external structured memory is. External structured memory is a family of architectures — represented in the 2026 literature by Zahn and Chana’s Knowledge Objects (KO) and by OIDA — in which knowledge is stored as typed structured units in an external store the LLM reads as a substrate client. KO encodes facts as hash-addressed Knowledge Objects with a `provenance_metadata` field, achieves $O(1)$ retrieval at near-constant per-query token cost, and demonstrates the database-like cost curve empirically — at $N=7,000$, $\sim 252\times$ lower per-query cost than in-context memory, with cost differences that “grow linearly as the corpus expands” (§6.2). OIDA structures organizational

knowledge as typed Knowledge Objects carrying epistemic class (fact, hypothesis, decision, question), importance scores with class-specific decay, and signed contradiction edges between conflicting objects, with a deterministic Knowledge Gravity Engine maintaining the substrate over time (§5.2, §8.2). The shared commitment across the family is that coordination-grade knowledge is a schema-level object with typed structure and maintained state, not a retrieval target recovered from a corpus at inference time.

The CKS-distinguishing commitment. CKS sits in the same architectural family as KO and OIDA — typed external substrate, separation of concerns between storage and processing, addressable units with provenance — and inherits the database-like cost model from this family (§6.2). What distinguishes CKS from the rest of the family is three architectural commitments developed in §5.2 against OIDA and a fourth axis surfaced in §6.2 against KO. *First*, human governance of the substrate as a design requirement: CKS commits to human authority over substrate content as a property of the architecture (§3.1), whereas KO and OIDA commit to maintenance architectures (KO’s hash addressing, OIDA’s Knowledge Gravity Engine) that operate over the substrate deterministically without locating authority in any particular actor. *Second*, formal role and authority schema at the cell level: CKS encodes who may decide what, on what evidence, under which policy, as substrate content the cell’s orchestration rules draw on; KO commits to a single `provenance_metadata` field, and OIDA’s Knowledge Objects carry epistemic class but not institutional provenance in the role-authority sense (§5.2). *Third*, the AI’s architectural role: CKS frames AI as mediator over the substrate (helping humans read, write, interpret, and compose substrate content, and executing cell operations under human-authored orchestration rules), whereas KO and OIDA frame AI as substrate client consuming structured memory to answer queries (§5.2, §8.2). *Fourth* — the axis specific to KO — where disambiguation work lives: KO disambiguates computationally at retrieval time (density-adaptive retrieval, learned thresholds, exact structured key matching on tuples); CKS disambiguates through human-authored schema at cell level, encoding distinctions into substrate structure at authoring time rather than recovering them by runtime algorithm (§6.2).

The conflation pattern. Two readings collapse the architectural difference into a feature delta. The first is “CKS is KO with extra fields.” The source paper preempts this directly at §6.2: the extra fields are not decoration; they are substrate content doing disambiguation work CKS’s design philosophy requires to be explicit rather than computed. The second is “CKS is OIDA plus multi-human use.” The source paper preempts this with a load-bearing phrase deployed at both §5.2 and §9.4: *CKS is architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human*. The three differentiation axes — governance, role/authority schema, AI’s architectural role — are independent commitments that together characterize a different design posture than the one KO and OIDA take, not increments on top of theirs.

One-sentence test. If the substrate’s design treats AI as a client consuming structured memory to answer queries, with no first-class commitment to human authority over substrate content or to a formal role/authority schema at the cell level, it is external structured memory of the KO/OIDA family; if the substrate is human-governed by architectural commitment, encodes role and authority semantics at the cell level, and treats AI as substrate mediator under human-authored orchestration rules, it is CKS.

5. Why naming these boundaries matters

CKS composes with each of the three adjacent objects in coherent hybrid systems, and the boundaries this note formalizes are what makes the composition possible without confusion. A CKS substrate may front a RAG retrieval index when source-document grounding is needed for content the substrate references — the substrate carries the decisions and rationale; the RAG layer supplies the source material those decisions rest on. A CKS cell may use an LLM mediator that has been fine-tuned on domain content — the mediator’s parametric memory improves its fluency in the domain, while the substrate remains the authoritative artifact for what has been decided in that domain. A CKS substrate may be composed with an external structured memory store of the KO or OIDA family — the structured memory holds typed facts and contradictions at scale, while the CKS layer adds the human governance, role/authority schema, and substrate-mediator semantics the structured memory does not commit to.

Each of these compositions is coherent only when the layers are named explicitly. A CKS-fronting-RAG hybrid becomes incoherent if the RAG layer is mistaken for the substrate, because governance is then read as retrieval policy rather than as authority over a coordination artifact. A CKS-with-fine-tuned-mediator hybrid becomes incoherent if the parametric memory is mistaken for the substrate, because expansion cost curves become unintelligible — the architecture appears to commit to linear-cost scaling while in fact carrying parametric expansion as a hidden term. A CKS-composed-with-KO hybrid becomes incoherent if the structured memory is mistaken for CKS, because the role/authority schema and the AI-as-substrate-mediator commitment disappear without anyone noticing, and the hybrid is then a KO deployment with the CKS layer reduced to decoration.

The boundaries this note formalizes are what allow the layers to be composed without collapsing into one another, and what allow downstream work to adopt CKS, vary it, compose it with adjacent patterns, or argue against it on its own architectural terms. None of the three adjacent objects is a competitor to CKS in the zero-sum sense; each is a different design object serving a different design goal, and the value of naming the boundaries is precisely that it lets all three coexist with CKS in the same system without ambiguity about which layer is doing what.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Related note in this series

Li, W. (2026). *Authority, Not Labor: A Precise Definition of “Human-Governed” in the Coordination Knowledge Substrate Pattern*. 24 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Three Adjacencies: Why CKS Is Not RAG, Not Parametric Memory, and Not External Structured Memory*. 24 April 2026. ORCID: 0009-0004-8065-3235.